

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 04 -

GERENCIADOR DE PACOTES COM HELM CHARTS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	24
O QUE VOCÊ VIU NESTA AULA?	24
REFERÊNCIAS.....	31

O QUE VEM POR AÍ?

Gerenciar manifestos YAML no Kubernetes manualmente, com várias versões para diferentes ambientes, é trabalhoso e propenso a erros. É nesse contexto que o **Helm** se destaca, atuando como um **gerenciador de pacotes** para Kubernetes.

Com o Helm, criamos **Charts** — pacotes reutilizáveis que permitem **templatização, parametrização e versionamento** das configurações. Isso evita a duplicação de código, facilita a manutenção e permite personalizar implantações por ambiente de forma **declarativa**.

Além disso, o Helm traz **controle de versões**, facilita **rollbacks**, e promove boas práticas como **infraestrutura como código, imutabilidade e idempotência** — garantindo consistência e previsibilidade nas automações.

Nesta aula, vamos entender como o Helm simplifica e padroniza implantações no Kubernetes, tornando o processo mais **eficiente, confiável e escalável**.

Vamos nessa?

HANDS ON

Vamos colocar a mão na massa construindo um Helm Chart do zero – e vamos além, utilizando a linguagem **Rust** como aliada para gerar, instalar e atualizar nosso chart.

Por que Rust? Porque, além do alto desempenho, Rust nos oferece segurança de memória e robustez na manipulação de arquivos YAML e templates declarativos, reduzindo riscos de erros sutis em arquivos de configuração. Em DevOps moderno, é cada vez mais comum usar ferramentas em Rust para garantir confiabilidade na automação, e aqui mostraremos esse potencial na prática.

SAIBA MAIS

Nesta seção, vamos apresentar um caso prático, e logo em seguida nos aprofundar em conceitos formais e teóricos por trás do uso de Helm e Infrastructure as Code, com direito a formalismo matemático e rigor computacional. Prepare-se, pois agora a conversa fica mais densa tecnicamente!

Implementando HELM Charts

Suponha que temos uma aplicação web escrita em Rust (pela performance e segurança, claro!) que queremos implantar no Kubernetes. Precisamos de um *Deployment* com algumas réplicas, um *Service* expondo a porta web e talvez um *ConfigMap* com configurações.

Faremos um Helm Chart parametrizado para essa aplicação, permitindo reutilizar o chart para diferentes versões ou ambientes apenas alterando valores, sem duplicar arquivos YAML.

Criaremos um diretório para o chart, por exemplo myrustapp-chart. Dentro dele, precisamos de pelo menos três coisas:

- Um arquivo `Chart.yaml` contendo metadados do chart (nome, versão do chart, versão do app, etc.).
- Um arquivo `values.yaml` com valores padrão configuráveis (por exemplo, imagem Docker padrão da nossa aplicação Rust, número de réplicas, porta).
- Um diretório `templates/` com os manifests Kubernetes em forma de templates Go.

Ao rodar `helm create myrustapp-chart`, o Helm gera uma estrutura básica típica:



Figura 1 - Estrutura típica de um Helm Chart, com templates e arquivos de valores
Fonte: elaborado pelo autor (2025)

Essa estrutura segue boas práticas: por exemplo, `_helpers.tpl` costuma conter *helpers* (templates parciais) para evitar repetir códigos de recurso (como construções de nomes), reforçando DRY. O arquivo `NOTES.txt` é opcional e serve para imprimir dicas após instalação (como comandos de acesso).

EDITANDO O CHART.YAML

Abra o `Chart.yaml` e edite os campos importantes:

```
apiVersion: v2          # apiVersion do formato de Chart (v2 é para Helm3)
name: myrustapp          # nome do chart
description: Chart de exemplo para app Rust
type: application        # chart de aplicação (não um chart dependência/library)
version: 0.1.0           # versão do chart
appVersion: "1.0.0"      # versão da aplicação Rust empacotada
```

Figura 2 – Chart.yaml
Fonte: elaborado pelo autor (2025)

Aqui definimos nossa primeira versão do chart (**0.1.0**) empacotando a versão **1.0.0** da aplicação (talvez correspondendo à tag Docker 1.0.0). O versionamento segue SemVer (X.Y.Z). Por convenção, atualizaremos `version` do chart ao fazer mudanças no próprio chart (ex.: template alterado ou valores padrão diferentes), enquanto `appVersion` acompanha a versão binária/imagem da aplicação Rust em si. Essa separação ajuda a governança, permitindo, por exemplo, evoluir o chart sem trocar a versão do app ou vice-versa.

Podemos usar um programa em Rust para manipular versionamento de forma segura. Por exemplo, uma ferramenta interna poderia ler o `Chart.yaml`, incrementar automaticamente o `patch version` antes de um release do chart, garantindo consistência. Usando a crate `serde_yaml`, é possível desserializar o YAML do Chart em uma struct Rust, alterar o campo de versão e salvar – evitando erros manuais de digitação. Isso mostra como Rust brilha ao lidar com infra declarativa: tipagem forte e segurança de memória ao nosso favor na automação.

DEFININDO VALUES.YAML

No arquivo de valores padrão, listamos configurações parametrizáveis do chart. Para nossa aplicação Rust, poderíamos ter algo assim em `values.yaml`:

```
replicaCount: 2

image:
  repository: myregistry/awesome-rust-app
  tag: "1.0.0"
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 80

resources:
  limits:
    memory: 256Mi
    cpu: 500m
  requests:
    memory: 128Mi
    cpu: 250m

# Habilitar ou não RBAC (caso nossa app precise roles, neste exemplo não precisa)
rbac:
  create: false
```

Figura 3 – values.yaml
Fonte: elaborado pelo autor (2025)

Aqui definimos que, por padrão, vamos rodar 2 réplicas (`replicaCount`) do nosso app Rust, usando a imagem `myregistry/awesome-rust-app:1.0.0`. Tornamos

a porta do *Service* configurável (80 por padrão), bem como limites de recurso. Esses valores podem ser todos sobrescritos pelo usuário no momento da instalação (`helm install`) via linha de comando ou via um arquivo YAML de override (com `-f custom-values.yaml`).

O importante é que **todas as configurações que provavelmente variam por ambiente** estejam externalizadas em `values.yaml`: por exemplo, em dev talvez `replicaCount: 1` e recursos mínimos, enquanto em produção `replicaCount: 5` e mais memória. Graças a essa parametrização, não precisamos manter manifestos separados – um único template gera manifestos distintos conforme os inputs fornecidos. Isso exemplifica o DRY: “cada pedaço de conhecimento deve ter uma representação única no sistema”, e aqui o “conhecimento” da configuração existe num só lugar (o chart), e não duplicado em vários YAMLS para cada ambiente.

CRIANDO TEMPLATES COM GO TEMPLATE

Agora edite `templates/deployment.yaml` para referenciar os valores acima usando a sintaxe de template do Helm (baseada em Go templates + funções Sprig). Um *Deployment* de exemplo poderia ser:


```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "myrustapp.fullname" . }}
  labels:
    app: {{ include "myrustapp.name" . }}
    chart: "{{ .Chart.Name }}-{{ .Chart.Version }}"
    release: "{{ .Release.Name }}"
    heritage: "{{ .Release.Service }}"
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ include "myrustapp.name" . }}
  template:
    metadata:
      labels:
        app: {{ include "myrustapp.name" . }}
    spec:
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - containerPort: {{ .Values.service.port }}
          resources:
            {{- toYaml .Values.resources | nindent 14 }}
```

Figura 4 – Template Helm
Fonte: elaborado pelo autor (2025)

Vamos dissecar alguns trechos importantes:

`{{ .Values.replicaCount }}` insere o valor configurado de réplicas. Se no `values.yaml` default é 2, e não for sobrescrito, o Deployment terá 2 réplicas. Se o usuário executar `--set replicaCount=5` no install, Helm renderá 5 aqui.

A notação `{{ .Values.image.repository }}` e `{{ .Values.image.tag }}` constrói o nome completo da imagem. Isso facilita a atualização de imagens: um `helm upgrade --set image.tag=1.1.0` pode apontar para uma nova versão do container Rust sem editar manifestos manualmente.

A função `include "myrustapp.fullname".` é um helper definido possivelmente em `_helpers.tpl` para construir nomes consistentes (geralmente concatenando

release name + chart name). Os helpers e templates parciais evitam repetição de lógica de string em múltiplos lugares.

Uso de `toYaml` e `nindent` para embutir sub-blocos YAML (aqui inserimos o yaml de recursos com indentação apropriada). Essas funções vêm da biblioteca **Sprig**, que adiciona mais de 60 funções utilitárias às templates Helm. Por exemplo, Sprig nos permite transformar strings, fazer cálculos, gerar strings aleatórias etc., tudo dentro do template.

Essas funções são **puras** no sentido computacional: recebem inputs e produzem um output determinístico sem efeitos colaterais. Isso está alinhado à ideia de que o template Helm pode ser visto como uma *função pura* que mapeia valores de entrada (`.Values`) para manifestos de saída. Trata-se basicamente de uma função `f(valores) -> YAML` que sempre que chamada com os mesmos valores gera o mesmo resultado, reforçando previsibilidade. Modelar templates dessa forma facilita a compreensão e teste – podemos rodar `helm template` (modo dry-run) para ver o output YAML, confiando que se não mudarmos os inputs, o resultado será idêntico.

Após escrever o template do Deployment, faríamos similar para `service.yaml`:

```
apiVersion: v1
kind: Service
metadata:
  name: {{ include "myrustapp.fullname" . }}
spec:
  type: {{ .Values.service.type }}
  ports:
    - port: {{ .Values.service.port }}
      targetPort: {{ .Values.service.port }}
      protocol: TCP
      name: http
  selector:
    app: {{ include "myrustapp.name" . }}
```

Figura 5 – Service.yaml
Fonte: elaborado pelo autor (2025)

Aqui, usamos `.Values.service.*` para configurar o tipo do serviço (ClusterIP, LoadBalancer, etc.) e a porta. Note que reutilizamos o mesmo label `app: myrustapp` tanto no Deployment quanto no Service, garantindo que o Service aponte para os Pods corretos. *Essa consistência vem “de graça” com templating*: definimos o label

uma vez (via helper `myrustapp.name` talvez) e usamos nos dois templates – sem risco de divergir. Novamente, sem Helm, um erro comum seria editar um YAML e esquecer de alterar o outro, quebrando o seletor.

TESTE DE TEMPLATE

Podemos agora testar se nossos templates estão corretos rodando:

```
helm install --dry-run --debug ./myrustapp-chart --set replicaCount=3
```

Figura 6 – Teste de template
Fonte: elaborado pelo autor (2025)

Isso fará o *render* do chart sem aplicá-lo no cluster, exibindo os YAML resultantes. No output, veremos o Deployment com replicas: 3 e demais campos preenchidos. Por exemplo, um trecho esperado do Service renderizado:

```
# Source: myrustapp-chart/templates/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: glowing-goose-myrustapp
spec:
  type: ClusterIP
  ports:
    - port: 80
      targetPort: 80
      protocol: TCP
      name: http
  selector:
    app: glowing-goose-myrustapp
```

Figura 7 – Service renderizado
Fonte: elaborado pelo autor (2025)

Repare que o Helm gerou um nome único `glowing-goose-myrustapp` combinando um identificador aleatório (release name gerado) com o nome do chart. Esse é o comportamento padrão se não especificarmos `--name`; em produção normalmente forneceríamos um nome fixo (ex: `helm install meuapp ./myrustapp-chart --namespace prod`). O importante é que o comando de *dry-run* confirma a

idempotência: se rodarmos de novo sem mudar nada, a saída será igual, e nos dá chance de inspecionar se tudo está correto antes de aplicar de fato no cluster.

INSTALAÇÃO DO CHART

Satisfeitos com os templates, vamos instalar de verdade. Supondo que você tenha acesso a um cluster Kubernetes (configurado no kubeconfig), rode:

```
helm install myrustapp-release ./myrustapp-chart --namespace dev
```

Figura 8 – Instalação do Chart
Fonte: elaborado pelo autor (2025)

Isso instala o chart no namespace dev com o release name myrustapp-release. O Helm irá:

- Comparar os manifests gerados com o estado do cluster (como não existe ainda, ele criará todos).
- Registrar essa *release* no cluster (Helm 3 armazena informações da release em um Secret no namespace, para rastrear histórico).

Se tudo der certo, o Helm imprimirá uma mensagem de sucesso juntamente com o conteúdo de NOTES.txt (que você pode personalizar para instruir, por exemplo, como acessar a aplicação).

A instalação via Helm é transacional e controlada: se algum recurso falhar ao criar, ele pode reverter o que foi feito (com flag `--atomic` por exemplo). Além disso, cada release recebe um número de revisão; se instalarmos novamente com o mesmo nome, Helm fará um upgrade ao invés de nova instalação.

ATUALIZAÇÕES (UPGRADE)

Vamos simular uma atualização de versão da aplicação Rust. Digamos que temos agora a imagem myregistry/awesome-rust-app:1.1.0 pronta para deploy com melhorias. Duas maneiras de atualizar:

- **Via linha de comando:** `helm upgrade myrustapp-release ./myrustapp-chart --set image.tag="1.1.0"`. Isso diz ao Helm para aplicar a nova

configuração (alterando apenas o tag da imagem). O Helm calculará o *diff* dos manifests (comparando com a revisão anterior) e aplicará somente as mudanças necessárias no cluster. Ferramentas como o plugin `helm diff` podem ser úteis para inspecionar essas diferenças antes de aplicar, uma boa prática para evitar surpresas.

- **Via versão do Chart:** Alternativamente, poderíamos codificar a nova versão de app no `values.yaml` (atualizar tag: 1.1.0) e incrementar a versão do Chart (`version: 0.1.1` por exemplo em `Chart.yaml`). Ao rodar `helm upgrade myrustapp-release ./myrustapp-chart`, Helm perceberá que o chart mudou de versão e aplicará mudanças (nova imagem) mantendo o histórico. Esse approach é útil se você está empacotando tudo para distribuir – geralmente se atualiza o `Chart.yaml` e cria um novo pacote (tgz) do chart.

Seja qual for o método, Helm **preserva o histórico de releases**. Podemos rodar `helm history myrustapp-release` e ver as revisões, com timestamps e status. Se algo der errado na versão nova (talvez o pod `CrashLoop` com a versão 1.1.0), podemos rapidamente acionar um **rollback**: `helm rollback myrustapp-release 1` (onde 1 é o número da revisão anterior estável). Isso retorna o cluster ao estado anterior usando os manifests antigos que Helm tem salvo, comprovando o benefício de versionamento e imutabilidade: não alteramos recursos "no lugar" manualmente, e sim declaramos um novo estado e deixamos o Helm orquestrar a transição, com opção de voltar atrás facilmente.

AUTOMATIZANDO COM RUST (OPCIONAL)

Vamos supor que queiramos automatizar parte desse ciclo com um utilitário escrito em Rust. Por exemplo, poderíamos criar um binário Rust que faça deploys condicionalmente. Com Rust, podemos usar a crate `kube` ou simplesmente chamar comandos do Helm via `std::process::Command`. Um pseudo-código Rust:

```
usestd::process::Command;
// ... obter novaVersao via argumento ou API ...
Command::new("helm")
    .args(&["upgrade", "myrustapp-release", "./myrustapp-chart",
            "--set", &format!("image.tag={}", novaVersao),
            "--atomic", "--cleanup-on-fail"])
    .status()
    .expect("Falha ao executar helm upgrade");
```

Figura 9 – Pseudo - código Rust
Fonte: elaborado pelo autor (2025)

Esse snippet ilustra como chamar o Helm para realizar um upgrade programaticamente. Podemos embutir lógica adicional em Rust, como por exemplo: realizar uma requisição HTTP de verificação de saúde após o upgrade e, em caso de falha, realizar rollback via outro comando `helm rollback`.

Assim construímos um controlador de deploy custom, utilizando o Helm para a parte de template/patch e Rust para lógica de alto nível. De fato, empresas como a Qovery optaram por integrar o Helm dentro de um mecanismo maior escrito em Rust, adicionando *lifecycles* customizados antes/depois do deploy, checagens adicionais. Rust garante que tais integrações rodem com alta performance e sem crashes inesperados, ideal para pipelines de CI/CD e operadores.

Outro uso de Rust seria gerar YAML de configuração em vez de usar templates. Um desenvolvedor poderia preferir definir recursos Kubernetes via structs em Rust (aproveitando crates geradas a partir do OpenAPI do Kubernetes para todos os tipos de recurso).

Assim, um programa Rust poderia: gerar manifestos completos a partir de dados de entrada e até aplicá-los via client Kubernetes – contornando o templating. Essa abordagem garante validação em tempo de compilação dos manifestos (você literalmente não consegue criar um YAML inválido se estiver fora do esquema, pois as structs Rust seguem o schema da API). É uma tendência interessante, embora *heavy-handed* para casos simples; Helm atinge um equilíbrio bom para a maioria das aplicações, mas saber que podemos usar Rust nesse contexto nos dá flexibilidade para cenários especiais e integrações de baixo nível.

IDEMPOTÊNCIA E IMUTABILIDADE EM INFRAESTRUTURA COMO CÓDIGO

Dois pilares fundamentais para garantir confiabilidade em automação de infraestrutura são **idempotência** e **imutabilidade**. Em termos matemáticos, uma operação f é *idempotente* se $f(f(x)) = f(x)$ para todo x . Ou seja, aplicar f duas, três ou n vezes produz o mesmo resultado que aplicar uma vez. Traduzindo para infraestrutura: uma definição de estado desejado deve poder ser aplicada múltiplas vezes sem mudar o estado já alcançado.

Helm e Kubernetes adotam essa filosofia declarativa – quando você executa `helm install` ou `helm upgrade`, o mecanismo subjacente da API do Kubernetes aplica somente as diferenças necessárias para atingir o estado final, sem recriar tudo do zero toda vez. Por exemplo, se já há 3 réplicas rodando e o chart deseja 3, nada acontece; se deseja 4, somente 1 réplica extra é criada.

Essa idempotência dá segurança para operações repetitivas ou automatizadas (você pode reexecutar pipelines sem medo de que “duplique” recursos). Conforme destacado por Harness: *“aplicar o mesmo código repetidamente resulta no mesmo estado final da infraestrutura”*. Assim, podemos rodar `helm upgrade` múltiplas vezes com o mesmo chart e valores, e após a primeira vez as subsequentes não provocarão mudanças – o cluster já está convergido naquele estado.

Imutabilidade refere-se a não modificar objetos existentes, mas sim substituí-los por novos quando há mudanças. Na infraestrutura tradicional, às vezes atualizava-se um servidor “em pé” (mudança in-place); na abordagem imutável, se precisamos alterar algo, criamos um artefato (uma nova máquina virtual, um novo contêiner) e apontamos o tráfego para ele, descartando o antigo.

No contexto do Helm/K8s, imutabilidade aparece, por exemplo, em *ConfigMaps* ou *Secrets* gerados por charts: uma prática recomendada é versionar no nome (como anexar um hash da configuração) para que, em vez de editar um ConfigMap, você crie um novo e atualize referenciadores. O Kubernetes faz muito isso internamente (Deployments imutáveis geram ReplicaSets diferentes a cada versão).

Charts bem desenhados abraçam a imutabilidade para facilitar rollbacks – ao fazer upgrade, não mutamos o objeto existente de forma irreversível, mas criamos nova versão e mantemos histórico. Isso complementa idempotência: podemos aplicar

o novo estado e caso precisemos reverter, aplicamos o estado antigo (que não foi alterado, pois estava guardado como histórico).

A idempotência garante repetibilidade de operações; imutabilidade garante reprodutibilidade de artefatos sem efeitos fantasmas de mudanças passadas. Formalmente, idempotência é uma propriedade de uma função de atualização ($f^n(x) = f(x)$) e imutabilidade é uma propriedade do modelo de estado (não há x que seja mutado; para mudar x , criamos x' independente). Ambos fornecem bases para **convergência determinística**: dado um target state, o sistema sempre alcança aquele state de forma consistente, não importando quantas iterações ou intervenções ocorram.

PARAMETRIZAÇÃO E DRY: TEORIA DA ABSTRAÇÃO DE CONFIGURAÇÃO

O **DRY (Don't Repeat Yourself)** é um clássico da engenharia de software que aplicado à infraestrutura significa evitar YAMLS duplicados para cada ambiente ou serviço. Formalmente, como dito no livro *The Pragmatic Programmer*: *"Every piece of knowledge must have a single, unambiguous, authoritative representation within a system"*. No nosso contexto, o “conhecimento” pode ser a definição de um Deployment.

Sem Helm, você talvez tivesse `prod-deployment.yaml`, `staging-deployment.yaml` etc., cada um repetindo 95% dos campos com pequenas diferenças (`replicas=3` vs `1`, imagem tag diferente, entre outras coisas). Isso viola DRY, pois esse *conhecimento* (o template do Deployment) está espalhado em vários lugares. Com Helm, extraímos esse conhecimento para uma única representação (um template paramétrico) e passamos *parâmetros* (`valores.yaml`) para acomodar variações.

Em termos teóricos, podemos modelar um template como uma função $T(p)$ onde p é um conjunto de parâmetros (valores). Sem templating, tínhamos múltiplas instâncias $Y_1, Y_2, \dots$ (YAMLS distintos) tais que $Y_1 = T(p_1)$, $Y_2 = T(p_2)$, mas escritos manualmente sem reconhecer a relação funcional entre eles. O templating torna essa relação explícita: definimos T uma vez, e para cada ambiente fornecemos p_i . Isso não é apenas conveniente – é matematicamente eliminar redundância e possibilitar ver T como objeto de composição. Além disso, evita inconsistências: se quisermos mudar algo estrutural (digamos, acrescentar um sidecar

no *Pod template*), fazemos na definição única `$T$` ao invés de 5 arquivos YAML diferentes. Este é o poder da abstração.

A parametrização, entretanto, tem seu custo cognitivo: exige que o autor do chart lide com a complexidade de generalizar a configuração. Em Helm, isso pode introduzir condicionais e lógica no template (por ex: `if .Values.featureX enabled then add section Y`), o que pode complicar a depuração. Mas mantido sob controle, o benefício supera o custo – especialmente com ferramentas de validação de valores (que veremos adiante).

Conceitos de teoria de linguagens de programação se aplicam aqui: um chart Helm é como uma função genérica, onde `values.yaml` são argumentos. O objetivo é que essa função seja a mais pura e determinística possível, para facilitar entendimento. É desejável evitar que templates Helm dependam de algo além dos inputs (por exemplo, acessar estado do cluster dentro do template – possível via função `lookup`, mas que traz efeito colateral e pode quebrar pureza).

As melhores práticas desencorajam lógica complexa nos templates, justamente para manter a analogia de *infraestrutura como código funcional*: sem efeitos colaterais, fácil de razonar, testável isoladamente (via `helm template`). Nesse sentido, a biblioteca Sprig que Helm usa provê funções matematicamente bem definidas (string ops, funções de lista, etc.) que ajudam a transformar dados de forma declarativa dentro do template.

TEMPLATES COMO FUNÇÕES PURAS E COMPOSIÇÃO DE CHARTS

Expanda a ideia acima: se um template é `$T(p) \to \text{YAML}`, então um Chart completo com múltiplos templates e valores, podem ser visto ver como um conjunto de funções que, juntas, produzem um conjunto de recursos.

Em Helm, podemos inclusive ter charts que **compõem** outros charts, os chamados *subcharts* ou dependências. Pense, por exemplo, em um *umbrella chart* que inclui um chart da aplicação Rust + um chart de banco de dados. O umbrella chart define talvez alguns valores globais e inclui os subcharts como componentes. Essa composição é análoga à composição de funções: podemos ter um chart `$C$` que recebe certos valores e internamente chama o chart `$A$` (aplicação) e `$B$` (banco)

com subconjuntos de valores. Formalmente, $C(v) = \{A(v_A), B(v_B)\}$ onde v_A , v_B são derivados de v . Assim como funções, charts podem ter **complexidade de composição**. Analisar a complexidade aqui não é sobre Big-O, mas sobre gerenciamento de configurações conforme gráficos de dependência crescem.

Uma preocupação prática é a **complexidade ciclomatica** do chart – quantos caminhos lógicos existem nos templates devido a condicionais. Charts muito genéricos podem virar um emaranhado de if/else para cada caso, difíceis de testar. É aqui que boas práticas entram: preferencialmente dividir em subcharts ou *library charts* pedaços reutilizáveis, ao invés de um único chart monolítico super flexível. Outra ajuda é o **schema de valores** introduzido no Helm 3. Podemos anexar um `values.schema.json` ao chart, definindo o tipo, default e restrições para cada campo esperado em values.

Isso torna explícito o “contrato” dos parâmetros e permite ao Helm validar os inputs antes de renderizar. Em termos de teoria de tipos, o schema age como um sistema de tipos para nossa função template: assegura que p pertence ao domínio válido de T . Se faltar um campo obrigatório ou tiver tipo errado, Helm aborta com erro em vez de gerar YAML potencialmente incorreto ou aplicar silenciosamente com comportamentos inesperados.

Essa validação de schema reduz a complexidade de depuração – ao invés de tentarmos descobrir por que um `.Values` não surtiu efeito (talvez porque estava mal escrito), receberemos um feedback imediato. Por exemplo, podemos exigir via JSON Schema que `.Values.replicaCount` seja inteiro ≥ 1 ; ou que `.Values.image.tag` siga um regex de versão.

Em termos computacionais, a **composição de charts e validação de schema** juntas fornecem modularidade e segurança: cada subchart é uma função com seu mini-esquema de inputs, e o chart pai monta inputs corretos para eles. Esse é um conceito análogo à composição de funções com checagem de tipos – se você conecta saída de uma função à entrada de outra, deve garantir que os tipos casem. O schema faz essa ponte. Assim, a engenharia de charts em escala envolve algo como *programação funcional declarativa*, onde charts são building blocks e o “código” que você escreve é na verdade YAML estruturado mais algumas expressões templates, mas o raciocínio pode ser tão rigoroso quanto em código normal.

GOVERNANÇA DE CHARTS: VERSIONAMENTO SEMÂNTICO, REPOSITÓRIOS DISTRIBUÍDOS E CONTROLE DE RELEASES (RBAC)

No mundo corporativo, além da parte técnica de templating, entra em cena a **governança**: como garantir que charts sejam confiáveis, seguros e gerenciáveis em larga escala. Vários aspectos teóricos e práticos se combinam aqui:

- Versionamento Semântico de Charts;
- Repositórios Distribuídos de Charts;
- Controle de Releases com RBAC;
- Algoritmos para Promoção Segura de Versões.

Assim como aplicações seguem o SemVer (X.Y.Z) para indicar compatibilidade, charts devem aderir ao **SemVer 2** conforme recomendado pelo Helm[\[6\]](#). Isso quer dizer: incrementos de *major* indicam mudanças incompatíveis (talvez removemos um campo de values ou mudamos comportamento padrão), *minor* para adição compatível e *patch* para correções/ajustes internos.

Essa disciplina permite automatizar pipelines – por exemplo, um pipeline de CI pode bloquear automaticamente uso de um chart major novo em produção sem revisão humana, pois sabe que major implica quebra potencial. SemVer nos charts facilita também a **promoção de versões**: podemos configurar deploy step de staging para aceitar upgrades até minor automaticamente, mas majors requerem intervenção. Em termos formais, temos uma relação de ordem parcial entre versões ($1.2.3 < 1.3.0 < 2.0.0$ etc.), e as políticas de promoção são definidas em relação a essa ordem.

Um *Helm repository* nada mais é que um servidor (HTTP ou armazenamento de objetos) servindo um índice (`index.yaml`) e pacotes `.tgz` de charts. Helm permite ter múltiplos repositórios configurados, incluindo repositórios públicos (como o Artifact Hub da CNCF) e privados (internos da empresa). Do ponto de vista de sistemas distribuídos, isso lembra *package managers* tradicionais (apt, npm): temos que lidar com questões de sincronização (atualizar índice quando publicar novo chart), distribuição geográfica, caching e segurança. A arquitetura do Helm 3 facilita isso pois não há um servidor central (Tiller) – o cliente Helm interage direto com repositórios e com o API server do cluster.

Assim, a distribuição de charts é fora do cluster, muitas vezes integrada com GitOps. Um padrão emergente é **usar Git como fonte de charts** – por exemplo, commitou no branch `main`, um action CI gera o pacote do chart e atualiza um repositório indexado (pode ser até o GitHub Pages servindo). O que é fundamental aqui é aplicar conceitos de **controle de versão distribuído** e **assinaturas digitais** para confiança: charts podem (e devem, em ambientes críticos) ser assinados e verificados na instalação. Helm suporta assinatura usando GPG e arquivos de proveniência (`.prov`) que acompanham o pacote.

Em termos de segurança computacional, isso se baseia em criptografia de chave pública e hashes – um chart assinado contém um hash SHA256 de seu conteúdo assinado com a chave privada do mantenedor, e o cliente Helm pode verificar com a chave pública correspondente. Isso garante **integridade e autoria** do pacote (supply chain security, alinhado ao modelo Zero Trust). Portanto, governança de charts envolve políticas como: *“somente instalar charts verificados de repositórios confiáveis”*.

No Kubernetes, todo acesso é mediado por RBAC (Role-Based Access Control). Isso significa que mesmo usando Helm, a capacidade de criar/atualizar recursos depende das permissões do usuário ou serviço executando o Helm. Diferente do Helm v2 (onde Tiller tinha seu próprio RBAC), no Helm v3 o cliente utiliza diretamente as credenciais do `kubectl`. Assim, um cluster admin pode granularmente controlar quem pode rodar `helm install` em certos namespaces ou quem pode atualizar certo tipo de recurso.

Por exemplo, você pode ter uma equipe de devs com role que permite operações somente no namespace `dev-*` e em recursos não privilegiados (Deployment, Service, etc.), mas não permite criar ClusterRole ou CRDs. Se eles tentarem instalar um chart contendo um ClusterRole (recurso cluster-wide), o Kubernetes negará a criação, mesmo que o Helm tente aplicar – resultando em erro de permissão. Isso é bom: evita que charts maliciosos ou mal configurados escalem privilégios inadvertidamente. Empresas devem definir **políticas RBAC** para Helm: por exemplo, um pipeline de CI rodando Helm para deploy pode ter um ServiceAccount com permissões restritas ao namespace da aplicação; se o chart tentar algo fora do escopo, será bloqueado (fail fast).

Do ponto de vista formal, RBAC em Kubernetes define conjuntos de permissão como tuplas (*subject, verb, resource, scope*). Um usuário ao chamar Helm executa na prática uma série de chamadas à API do K8s (verbs como create, patch, delete em certos resources). O Kubernetes fará matching entre essas ações e as regras RBAC. Assim, podemos modelar: *HelmRelease* é permitida se $\forall$ ação $\$in\$$ {ações do Helm} existe uma regra autorizando. Se $\exists$ ação não autorizada, a transação falha.

Em muitos casos, charts oficiais incluem templates para criar Roles/RoleBindings necessários para a própria aplicação rodar, e normalmente permitem desligar via `.Values.rbac.create=false` se o cluster já prover (conforme visto antes). Essa é outra faceta: charts podem *embutir recursos RBAC* para a aplicação, o que também deve estar sob controle. A boa prática é que charts segurem as mínimas permissões necessárias – ex: o chart do *Falco sidekick* cria um Role com escopo limitado apenas a ler Endpoints no namespace, em vez de pedir cluster-admin. O formalismo aqui é princípio de menor privilégio, implementado por construção de conjuntos de permissões mínimos no template.

Em ambientes enterprise, não basta ter charts – é preciso orquestrar como novas versões de charts (ou de valores) fluem do desenvolvimento até produção de forma **segura e automática**. Podemos pensar em um *pipeline algorítmico de promoção*. Imagine uma sequência de ambientes [Dev → QA → Prod]. Queremos um algoritmo que, dada uma nova versão de chart, realize:

- Implantar em Dev; se sucesso (pods saudáveis, testes passando), então
- Promover para QA; se métricas X e Y ok após T horas, então
- Promover para Prod; caso contrário, rollback ou não promover.

Isso é um fluxo de trabalho que pode ser descrito em pseudocódigo:

```
forenvin [Dev, QA, Prod]:
  deploy(chartVersion, env)
  result=runTests(env)
  if !result.success:
    haltPipeline()
    rollback(env)
    break
  // optionally wait and verify metrics in production stage
endfor
```

Figura 10 – Fluxo de trabalho
Fonte: elaborado pelo autor (2025)

Ferramentas de GitOps e CI/CD implementam variações disso. Por exemplo, o Codefresh Helm Board permite “arrastar” uma release de uma coluna (ambiente) para outra – por trás dos panos, isso dispara um pipeline que executa `helm upgrade` no próximo cluster/namespace e registra o evento. Há algoritmos de **canary deployments** onde em vez de ir 0%→100% existe um incremento gradual: e.g., implantar versão nova para 10% dos pods, monitorar, depois 50%, até 100%. Isso pode ser controlado por tags de versão no chart, ou até dois Helm releases simultâneos com divisões de tráfego via Service.

São estratégias algorítmicas de **promoção segura**, inspiradas em controle de versão de software e rollout de features. A teoria de controle entra aqui: tratar a implantação como um sistema que precisa de feedback (monitoramento) para determinar próximo passo – um loop do tipo *open feedback loop* onde métricas definem se promove ou reverte. Helm em si fornece ferramentas básicas (histórico, rollback, testes integrados via hooks), mas a lógica de alto nível fica a cargo do usuário/pipeline.

Com Rust ou outras linguagens, podemos codificar essa lógica: por exemplo, um controlador escrito em Rust observa repositórios de charts e automaticamente incrementa implantações nos clusters seguindo um algoritmo de promoção, garantindo que nunca mais que X% dos ambientes estejam rodando versão não validada. Podemos até aplicar conceitos formais como **grafos de dependência** de versões (somente promover se dependências correlatas também promoveram suas versões compatíveis).

Outro aspecto é **algoritmos de resolução de dependências**: se um chart A depende de chart B (por exemplo, uma aplicação que depende de um database chart), ao promover A, precisa garantir B está presente em versão compatível. Helm permite declarar dependências e fixar versões mínimas no `Chart.yaml`. O resolver de dependências do Helm (com `helm dependency update`) busca as versões corretas conforme especificadas.

Assim, a promoção segura de A envolve também garantir que B atenda às restrições. Isso é similar ao problema de satisfatibilidade de versões – pode ser tratado com algoritmos tipo backtracking, mas no escopo de charts geralmente é simples

(especificamos exatamente ou um range). A formalização disso entra na teoria de versões – semântica de compatibilidade codificada em números, que computacionalmente vira comparações.

A governança de charts combina princípios de engenharia de software (versionamento semântico), ciência da computação (algoritmos de deploy/teste/rollback, verificação de assinatura) e administração de sistemas (RBAC, segregação de ambientes). Todos esses elementos teóricos garantem que, ao adotar Helm em escala, tenhamos um processo previsível, verificável e seguro para lançar mudanças de infraestrutura e aplicações.

MERCADO, CASES E TENDÊNCIAS

Depois de tanto rigor, vamos descontrair um pouco e conversar sobre **casos de uso reais (e alguns fictícios)** de Helm no mundo corporativo e tendências que estão emergindo. Aqui vale uma pitada de storytelling lúdico, analogias criativas e até algumas figuras para ilustrar ideias.

CASO 1: A SAGA DO DEPLOY NA FINTECH REGULADA

Imagine a FinCorp, uma fintech operando em múltiplos países. Eles têm clusters Kubernetes separados por região, cada um com requisitos regulatórios (por exemplo, certos logs precisam ser retidos, certos serviços extras de compliance rodando). Inicialmente, cada time regional fazia seus manifestos – um pesadelo para o time central de SRE: configurações divergentes e difícil garantir que todos seguem políticas corporativas.

Entra o Helm: o time central desenvolve um **chart umbrella “fintech-app”** que encapsula as melhores práticas e componentes obrigatórios (monitoramento, sidecars de compliance). Cada região usa esse chart, passando valores específicos (URLs de endpoints locais, tamanhos de node diferentes). Resultado? Implantação padronizada, mas com flexibilidade local. Quando a regulação muda – digamos, precisar rodar um novo agente de segurança – basta atualizar o umbrella chart (nova versão 2.0.0) incluindo o agente, e todas as regiões fazem upgrade seguindo semver (major update requer testes em staging).

Essa história exemplifica como Helm facilita **governança centralizada com execução descentralizada**. Uma analogia: é como uma franquia de restaurante – o cardápio (chart) é o mesmo para todas as unidades, garantindo qualidade e conformidade, mas cada unidade pode ajustar detalhes (valores) como tempero ao gosto local. E se der algo errado em alguma filial (release com problema), conseguem reverter ao menu anterior rapidamente (rollback). FinCorp dorme tranquila e os auditores também, pois cada release Helm gera trilhas auditáveis (quem lançou qual versão, quando), útil para compliance financeiro.

CASO 2: HELM NO EDGE – A ODISSEIA DOS DISPOSITIVOS

Uma história fictícia: a DroneFleet Inc. gerencia 5000 drones autônomos que executam cargas de trabalho em clusters Kubernetes embarcados (sim, mini clusters voadores!). Eles usam Helm para atualizar o software nesses drones remotamente. Cada drone reporta a sua “versão de chart” atual. Quando sai um update, um sistema central (escrito em Rust, claro, pela eficiência) orquestra gradualmente a atualização: primeiro 1% dos drones (fase canário), monitora via telemetria se tudo bem (taxa de erro de rede, consumo de bateria).

Se aprovado, vai para 10%, e assim por diante. Esse sistema utiliza **Helmfile/Helm + Rust** para aplicar charts em batch e avaliar respostas. Visualize um painel com um *mapa de calor* dos drones – verde significa chart v3.2 está rodando, amarelo significa atualizando, vermelho se falha. Graças ao Helm, atualizar não requer logar em cada drone; é literalmente um comando automático que aplica o chart predefinido (que contém Deployments para componentes do drone).

A **tendência** aqui é Helm saindo do datacenter e indo para a borda (edge computing), onde clusters Kubernetes estão em fábricas, veículos autônomos, etc. Os princípios de padronização e declaração que funcionam no cloud valem ouro no edge, onde escalabilidade e desconexões temporárias são desafios. Uma figura mental: Helm Charts voando junto com drones, entregando “pacotes de atualização” tal qual pacotes de correio.

CASO 3: CI/CD GITOPS COM HELM – O CONTO DO PIPELINE AUTOMATIZADO

Considere a SoftServe Corp, que adotou GitOps. Todo código (incluindo charts Helm) reside em repositórios Git. Um desenvolvedor faz merge de uma nova feature que requer uma alteração de configuração. Ao invés de editar arquivos de manifesto diretamente, ele atualiza um **arquivo de values** específico do ambiente staging no repositório GitOps.

Um agente como Argo CD detecta a mudança e, usando Helm, faz o *sync* no cluster staging. Se a aplicação sobe ok e os testes de integração (definidos como hooks de Helm) passam, o pipeline automaticamente abre uma Pull Request para promover a mesma mudança ao arquivo de produção. Após revisão e merge, Argo

CD atualiza prod via Helm. Essa história mostra a sinergia de Helm com GitOps: charts fornecem modularidade e repetibilidade, Git fornece histórico e gating das mudanças.

A **tendência** aqui é forte: muitos times usam *Helmfile*, *Argo CD Applications* ou *Flux Helm Controller* para gerenciar centenas de releases Helm de forma declarativa e audível. Em termos lúdicos, GitOps com Helm é como ter um **maestro robótico**: o repositório Git é a partitura musical, Helm é a orquestra executando as notas no Kubernetes. Cada commit no Git aciona uma nova sinfonia de comandos `helm upgrade` nos clusters, mantendo tudo em harmonia. E se uma nota falsa soar (um deploy quebrar), o maestro nota e volta para partitura anterior (rollback), mantendo o espetáculo rodando sem grandes interrupções.

CASO 4: DEVSECOPS E HELM – PROTEGENDO A CADEIA

A empresa SecureTel adotou DevSecOps e examina cada etapa do ciclo de vida por potenciais riscos. Eles usam Helm extensivamente, então concentraram esforços em:

- **Scan de charts:** Integrar ferramentas que varrem charts em busca de más configurações – portas abertas, permissões amplas. Por exemplo, a pipeline de CI roda `helm lint` e também verifica se não há `ClusterRole` sendo criado sem necessidade. Qualquer chart que tente criar algo arriscado falha no build.
- **Assinatura e política:** Todos os charts promovidos a produção são assinados digitalmente pelo time de SRE. O cluster Kubernetes tem um *Admission Controller* (OPA Gatekeeper, por exemplo) que verifica, no ato de `helm install/upgrade`, se o chart (como pacote) está assinado e se o certificado é da SecureTel. Se alguém tentar instalar um chart não aprovado (não assinado), é barrado – impedindo que engenheiros desavisados implantem algo de fontes não confiáveis, prevenindo *supply chain attacks*. Essa abordagem se alinha com iniciativas como Sigstore Cosign, embora no Helm se use geralmente PGP ou Key Management interno.
- **Least Privilege por padrão:** Os charts mantidos pela org seguem uma cartilha: sempre incluir templates de RBAC para dar apenas acesso

necessário à aplicação. O time de segurança revisa esses templates. Assim, se um desenvolvedor subir um microserviço novo e usar um Chart Helm comum da empresa, pode ter certeza que ele não rodará com privilégios demais. Isso mitigou enormemente riscos – antes, alguns deployments eram aplicados manualmente com conta de admin e rodavam como root, agora os charts já forçam a criação de ServiceAccounts dedicadas, Roles limitadas e até controles como PodSecurityPolicy (ou equivalente) embutidos.

- **Auditoria contínua:** Como cada helm release deixa rastro (com quem executou, e os manifests ficam registrados no revision history), a SecureTel periodicamente audita diferenças entre releases para ver mudanças suspeitas. É quase como verificar commits de código, mas para infraestrutura. Se notarem que um Deployment de repente tem ambiente `DEBUG=true` ou volume mount estranho, investigam o release que introduziu aquilo.

A **tendência** aqui é que Helm não é apenas uma ferramenta de DevOps, mas também uma superfície de segurança a ser endereçada. Já existe até pesquisa em andamento sobre vulnerabilidades em charts públicos e maneiras de mitigá-las. A comunidade criou ferramentas para scanning automatizado de charts em busca de padrões perigosos. Em paralelo, o Helm tem evoluído: por exemplo, no futuro poderemos ver integração nativa com Sigstore (já há plugin) e políticas de verificação mais ricas.

CASO 5: ALÉM DO HELM – NOVOS RUMOS?

Por fim, é interessante notar tendências que questionam ou complementam o Helm. Lembra da seção anterior sobre usar Rust para gerar YAML? Pois é, há um movimento surgindo de *Configuration as Code* total, usando linguagens generalistas (como Python, Rust, CUE Lang) para gerar configurações Kubernetes, em vez de DSLs de templating. A justificativa: melhor controle, teste unitário de configurações (você pode escrever testes em Rust para checar que seu output YAML atende requisitos).

Ferramentas como **Kustomize** coexistem com Helm, oferecendo abordagem sem templating (patches declarativos); algumas empresas usam *Helm* para empacotar *Kustomize*, ou vice-versa. E no horizonte desponta a ideia de **GitOps 2.0 com IA** – imagine sistemas inteligentes sugerindo atualizações de charts, ou detectando automaticamente parâmetros ótimos. Já tem artigo propondo *AI-driven configuration generation*. Pode soar futurista, mas quem sabe um dia você descreve em linguagem natural “quero implantar 3 cópias desse serviço com 2GB de memória cada, escalonáveis até 5” e uma IA gera o `values.yaml` correspondente.

No curto prazo, Helm permanece líder por sua comunidade enorme (75% adoção segundo CNCF 2023) e suporte de ferramentas. A tendência concreta é **Helm + GitOps + Policy**. Em outras palavras, usar Helm para o que ele é bom (pacotes configuráveis), Git como fonte de verdade, e políticas automatizadas para garantir qualidade (lint, test, verify).

Essa tríade endereça os medos iniciais que alguns tinham de Helm (do tipo “é perigoso aplicar coisas templadas cegamente”). Hoje, boas práticas mitigam isso e Helm continua a facilitar a vida em casos novos – seja deploy contínuo em edge, seja fornecer fácil “uninstall” de aplicações complexas (Helm guarda a lista de tudo que criou, então `helm uninstall` remove limpo, recurso por recurso, coisa difícil se você aplicou dezenas de YAMLS manualmente).

Para fechar esta seção com uma imagem mental divertida: se Kubernetes é o sistema operacional do datacenter, **Helm é o gerenciador de pacotes** dele – como um *apt-get* ou *yum* do mundo cloud-native. E estamos vendo esse “gerenciador de pacotes” evoluir, ficar mais seguro e integrado. Assim como num OS você confia que os pacotes oficiais são verificados e mantém seu sistema consistente, no Kubernetes confiamos que charts bem-feitos nos dão aplicações estáveis e fáceis de administrar.

E assim a história do Helm continua navegando pelos mares (já que **Helm** também significa timão de navio) da computação em nuvem, guiando as implantações através de águas calmas e tempestades com a mesma tranquilidade – pois foi construído com sólidos princípios computacionais e melhorado com as lições da indústria.

O QUE VOCÊ VIU NESTA AULA?

Ufa! Vamos recapitular de forma leve os principais aprendizados desta aula densa sobre Helm Charts:

Helm ao resgate do YAML duplicado: Entendemos os problemas de gerenciar manifestos Kubernetes manualmente e como o Helm atua como um **gerenciador de pacotes**, empacotando múltiplos YAMLs em charts reutilizáveis, com suporte a *templates* e *valores* para evitar repetição. Vimos que isso traz reusabilidade, padronização e versionamento – em suma, menos erros e implantações mais consistentes.

Mão na massa com Helm e Rust: Passo a passo, criamos um chart para uma aplicação Rust, explorando `Chart.yaml`, `values.yaml` e `templates` de `Deployment/Service`. Praticamos comandos `helm install`, `helm upgrade`, `helm rollback` e entendemos o fluxo de trabalho do Helm. Além disso, justificamos o uso de **Rust** como ferramenta DevOps: seja para manipular arquivos YAML de forma segura ou orquestrar chamadas do Helm programaticamente, ganhando desempenho e confiabilidade extra (afinal, nada de *segfaults* ou *memory leaks* no nosso helper de deploy!). Você viu exemplos de código/estrutura mostrando como Rust pode incrementar automação em torno do Helm.

Conceitos avançados e teoria: Mergulhamos nos fundamentos de idempotência (aplicar várias vezes é seguro) e imutabilidade (não alterar no lugar, mas sim substituir), vital em infraestrutura declarativa. Debates o princípio DRY e como parametrização de charts é um caso de abstração funcional – templates Helm como se fossem funções puras transformando inputs em configurações, auxiliadas por funções Sprig. Falamos também de composição de charts e validação de schema, trazendo um quê de “sistema de tipos” para nossos valores, prevenindo erros bobos de configuração antes mesmo do deploy. Tudo isso com formalismos onde coube, para mostrar que, sim, por trás das YAMLs existe teoria de computação interessante!

Governança e algoritmos de deploy: Discutimos como charts são versionados semanticamente (e por quê), como distribuí-los via repositórios (públicos tipo ArtifactHub ou privados) e como controlar releases com RBAC – garantindo que só quem pode, faz, e dentro dos limites permitidos. Exploramos o conceito de pipelines

de promoção segura, canary releases e automação GitOps integrando Helm. Em resumo, vimos que gerenciar charts em larga escala requer processos e até algoritmos bem pensados para evitar caos – e Helm fornece hooks e ferramentas para implementar essas estratégias.

Histórias e tendências do mundo real: Para adoçar a aula, passemos por casos de uso: desde fintechs usando charts para padronizar deploys multi-região, passando por drones no edge recebendo updates via Helm, até empresas focadas em DevSecOps integrando assinaturas digitais e scans de segurança nos charts. Identificamos tendências como a combinação de Helm com GitOps e políticas de segurança, e apontamos curiosidades do futuro (IA configurando infra? Charts gerados puramente por código?). Tudo para ilustrar que Helm não vive isolado – ele está no centro de muitas práticas modernas e continua evoluindo conforme a orquestração de contêineres amadurece.

Nesta aula, você percorreu todos os aspectos do **Helm Charts** em profundidade: do básico ao avançado, da prática à teoria, do presente ao futuro. Agora você não apenas sabe usar o Helm, mas entende os *porquês* de suas práticas e como encaixá-lo num cenário maior de DevOps/Kubernetes avançado.

Parabéns por chegar até aqui – que esse conhecimento lhe dê ferramentas para navegar no mar de YAMLS e deployments com muito mais tranquilidade e controle (apenas não esqueça de usar `helm diff` antes de instalar algo em produção, hein?).

REFERÊNCIAS

ANARCHER. A toy tool for generating Kubernetes manifests in Rust. 2025. Disponível em: <https://anarcher.blog>. Acesso em: 2 set. 2025.

BITNAMI (VMware). Understand a Helm chart structure. Bitnami Kubernetes Documentation. 2025. Disponível em: <https://docs.bitnami.com/kubernetes/how-to/understand-helm-chart-structure/>. Acesso em: 2 set. 2025.

CODEFRESH.IO. Promoting Helm Environments. Documentação. Disponível em: <https://codefresh.io/docs/docs/helm/promoting-environments/>. Acesso em: 2 set. 2025.

CNCF (Cloud Native Computing Foundation). CNCF Survey 2023 – dados de adoção do Helm. 2023.

HARNES.IO. How to Implement Infrastructure as Code: A Comprehensive Guide. 2025. Disponível em: <https://harness.io/blog/infrastructure-as-code-guide/>. Acesso em: 2 set. 2025.

HELM.SH. Role-based Access Control - Helm (Helm v3 docs). Disponível em: <https://helm.sh/docs/topics/rbac/>. Acesso em: 2 set. 2025.

HUNT, A; THOMAS, D. The Pragmatic Programmer. Boston: Addison-Wesley, 1999.

KUŠEN, B. Mastering Helm Security in 2024. Opstergo Blog. 2023. Disponível em: <https://opstergo.blog/helm-security-2024>. Acesso em: 2 set. 2025.

MAVRO, P. How we extended Helm lifecycle with Rust. Qovery Engineering Blog. 2021. Disponível em: <https://engineering.qovery.com/post/helm-lifecycle-rust/>. Acesso em: 2 set. 2025.

MINGUEZ, E. Helm security and best practices. Sysdig Blog. 2022. Disponível em: <https://sysdig.com/blog/helm-security-best-practices/>. Acesso em: 2 set. 2025.

REDDIT – DevOps. Quais são suas melhores práticas para implantar gráficos Helm? 2023. Disponível em: <https://www.reddit.com/r/devops>. Acesso em: 2 set. 2025.

SCHMITT, J. What is Helm? A complete guide. CircleCI Blog. 2025. Disponível em: <https://circleci.com/blog/what-is-helm/>. Acesso em: 2 set. 2025.

SOUZA, A. B. Primeiros passos com Helm no Kubernetes. DevSecOps from Zero! – Medium. 2022. Disponível em: <https://medium.com/devsecops-from-zero/helm-no-kubernetes>. Acesso em: 2 set. 2025.

SUSNJARA, S.; SMALLEY, I. What is Helm? IBM Think 2025. Disponível em: <https://www.ibm.com/blog/what-is-helm>. Acesso em: 2 set. 2025.

TOWARDSDEV. Rethinking Configuration — Beyond Declarative DSLs In Kubernetes. Medium. 2023. Disponível em: <https://medium.com/towardsdev/kubernetes-configs>. Acesso em: 2 set. 2025.

EMENDAS

PALAVRAS-CHAVE

Kubernetes. Rollout. Canary.

EMANIP

The background is a dark, textured surface with a network of thin, light teal lines and small circular nodes. These lines and nodes are scattered across the page, creating a sense of digital connectivity and data flow. Some lines are vertical, while others are diagonal or horizontal, connecting various points. The overall aesthetic is futuristic and technological.

POSTECH